EXPERIMENT 6: BIDIRECTIONAL RNN VS FEEDFORWARD NN FOR TIME-SERIES PREDICTION

Aim:

To implement a Bidirectional Recurrent Neural Network (BiRNN) for time-series prediction using the Airline Passenger Dataset and compare its performance with a Feedforward Neural Network (FFNN) using Mean Squared Error (MSE).

Procedure

1. Import the required libraries for data handling, model building, evaluation, and plotting.

2. Load the monthly Airline Passenger Dataset and normalize the passenger counts using MinMaxScaler.

3. Create input sequences using the previous 10 months to predict the next month's passenger count.

4. Split the data into training and testing sets in chronological order.

5. Convert the data into PyTorch tensors and prepare a DataLoader.

6. Define the Bidirectional RNN and Feedforward Neural Network architectures.

7. Train both models using the Adam optimizer and Mean Squared Error loss.

8. Generate predictions for the test set and convert the values back to the original scale.

9. Calculate and compare the MSE values and plot actual versus predicted passenger counts.

Cell 1 – Import libraries and load the Airline Passenger Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/refs/heads/master/monthly-airline-
passengers.csv"
df = pd.read_csv(url, usecols=[1])
data = df.values.astype("float32")

scaler = MinMaxScaler(feature_range=(0, 1))
data_scaled = scaler.fit_transform(data)
```

```
      Month  Passengers
0  1949-01         112
1  1949-02         118
2  1949-03         132
3  1949-04         129
4  1949-05         121
Dataset shape: (144, 2)
Dataset loaded and normalized successfully
```

Cell 2 – Create sequences and prepare training/testing data

```
SEQ_LENGTH = 10

X = np.array([
    data_scaled[i:i + SEQ_LENGTH]
    for i in range(len(data_scaled) - SEQ_LENGTH)
])
y = np.array([
    data_scaled[i + SEQ_LENGTH]
    for i in range(len(data_scaled) - SEQ_LENGTH)
])

train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)

train_loader = DataLoader(
    TensorDataset(X_train_tensor, y_train_tensor),
    batch_size=16,
    shuffle=True
)
```

```
Training samples: 107
Testing samples: 27
Input shape: torch.Size([107, 10, 1])
```

Cell 3 – Define BiRNN and Feedforward Neural Network models

```
class BiRNN(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, num_layers=1):
        super(BiRNN, self).__init__()
        self.rnn = nn.RNN(
            input_size,
            hidden_size,
            num_layers,
            batch_first=True,
            bidirectional=True
        )
        self.fc = nn.Linear(hidden_size * 2, 1)

    def forward(self, x):
        out, _ = self.rnn(x)
        return self.fc(out[:, -1, :])


class FeedforwardNN(nn.Module):
    def __init__(self, input_size):
        super(FeedforwardNN, self).__init__()
        self.fc1 = nn.Linear(input_size, 64)
        self.relu = nn.ReLU()
```

```
        self.fc2 = nn.Linear(64, 1)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        return self.fc2(self.relu(self.fc1(x)))
```

```
BiRNN(
  (rnn): RNN(1, 16, batch_first=True, bidirectional=True)
  (fc): Linear(in_features=32, out_features=1, bias=True)
)
```

Cell 4 – Train both models

```
def train_model(model, loader, epochs=100):
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

    for epoch in range(epochs):
        total_loss = 0.0
        model.train()

        for sequences, targets in loader:
            optimizer.zero_grad()
            outputs = model(sequences)
            loss = criterion(outputs, targets)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        if (epoch + 1) % 20 == 0:
            print(
                f"Epoch {epoch + 1}/{epochs}, "
                f"Loss: {total_loss / len(loader):.5f}"
            )


birnn = BiRNN()
train_model(birnn, train_loader)

ffnn = FeedforwardNN(input_size=SEQ_LENGTH)
train_model(ffnn, train_loader)
```

```
Training Bidirectional RNN...
Epoch 20, Loss: 0.017171
Epoch 40, Loss: 0.004673
Epoch 60, Loss: 0.003710
Epoch 80, Loss: 0.003440
Epoch 100, Loss: 0.003314

Training Feedforward Neural Network...
Epoch 20, Loss: 0.011405
Epoch 40, Loss: 0.004759
Epoch 60, Loss: 0.003704
Epoch 80, Loss: 0.002813
Epoch 100, Loss: 0.002362
```

Cell 5 – Predict and compare MSE

```
birnn.eval()
ffnn.eval()

with torch.no_grad():
    pred_birnn = birnn(X_test_tensor).numpy()
    pred_ffnn = ffnn(X_test_tensor).numpy()

pred_birnn_inv = scaler.inverse_transform(pred_birnn)
pred_ffnn_inv = scaler.inverse_transform(pred_ffnn)
y_test_inv = scaler.inverse_transform(y_test)

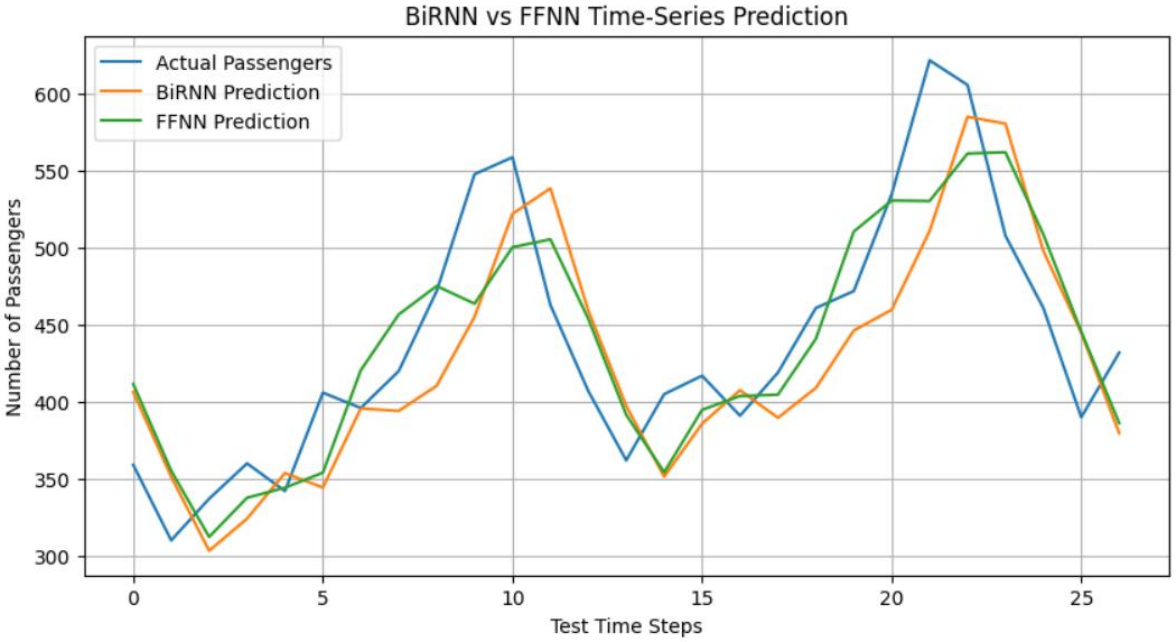
mse_birnn = mean_squared_error(y_test_inv, pred_birnn_inv)
mse_ffnn = mean_squared_error(y_test_inv, pred_ffnn_inv)

print(f"BiRNN MSE: {mse_birnn:.3f}")
print(f"FFNN MSE: {mse_ffnn:.3f}")
```

```
BiRNN Mean Squared Error: 2725.9053
FFNN Mean Squared Error: 1930.7316
FFNN performed better on this test set.
```

Cell 6 – Plot actual and predicted passenger counts

```
plt.figure(figsize=(12, 5))
plt.plot(y_test_inv, label="Actual")
plt.plot(pred_birnn_inv, label="BiRNN Prediction")
plt.plot(pred_ffnn_inv, label="FFNN Prediction")
plt.legend()
plt.title("BiRNN vs Feedforward NN on Airline Passenger Data")
plt.xlabel("Time Step")
plt.ylabel("Passengers")
plt.show()
```



RESULT

The Bidirectional RNN and Feedforward Neural Network were implemented and trained on the Airline Passenger Dataset. Their predictions were plotted against the actual passenger counts, and their MSE values were calculated for comparison. Record the MSE values and observations produced by your own run.